

Part I:

Public link to Edge Impulse project: <https://studio.edgeimpulse.com/public/1011573/live>

Lift State:

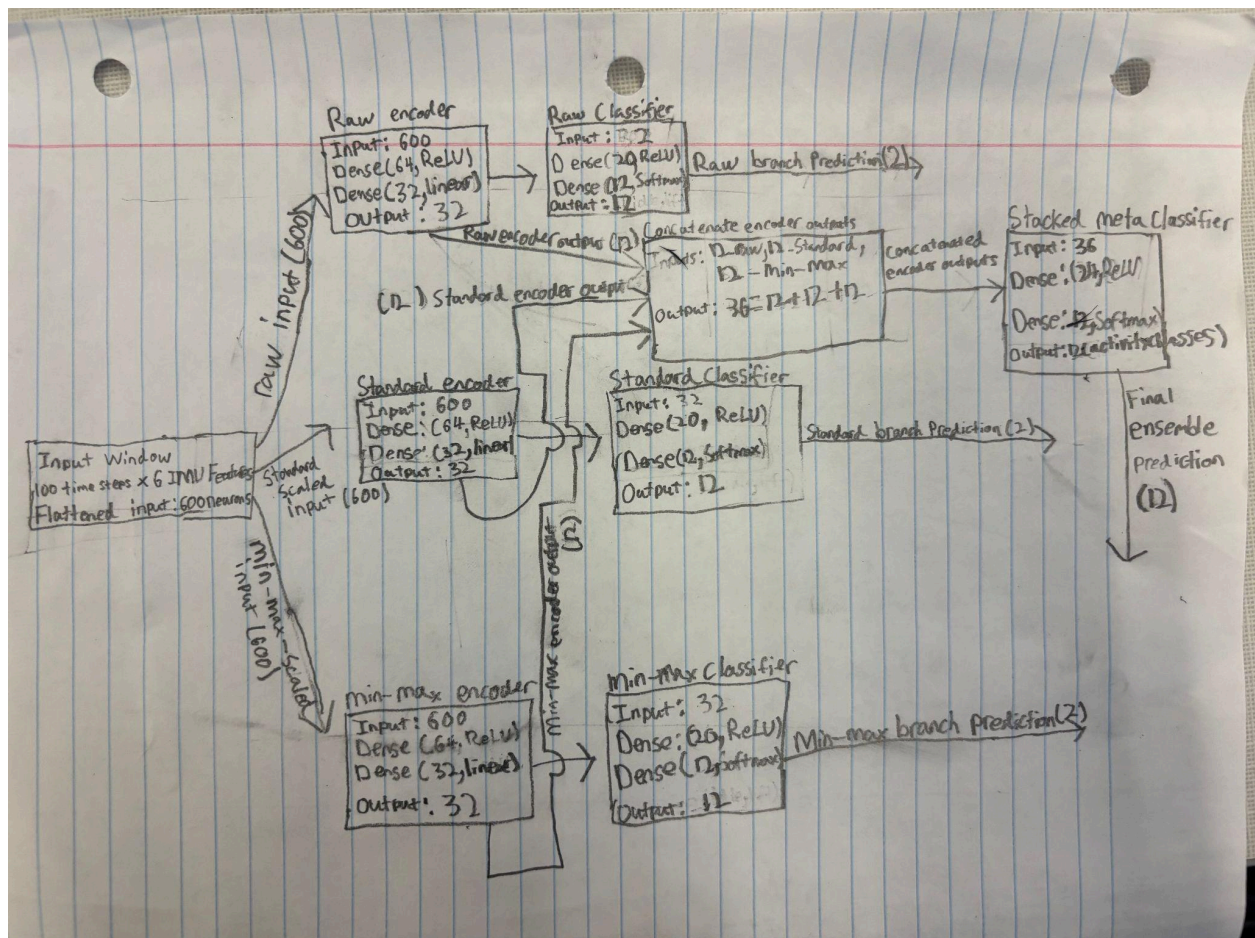
```
Sampling...
Timing: DSP 159 ms, inference 569 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
```

Idle State:

```
Sampling...
Timing: DSP 159 ms, inference 569 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
```

Part II:

1.



2.

The notebook compresses the TinyML models in three stages: pruning, QAT, and full-integer TFLite conversion.

Firstly, the layer sets are pruned using low magnitude pruning with a polynomial decay starting from 10% sparsity to reach 80% in 500 steps and taking 5 epochs for the model to train with this pruning wrapper applied.

Next, after applying pruning and evaluating it, the pruning wrappers are removed from the model and the result is the same bare sparse model required by quantization techniques due to its simple structure.

Quantization aware training is performed using wrapping this sparse model in quantization layers and training it for 5 epochs in order to teach it how to deal with int8 quantization noise.

Furthermore, binary masks are extracted from the pruned model and used by applying a callback function such that the model can get these masks re-applied both during the quantization aware training and right after completing it.

Finally, quantized models are transformed into int8 models in TFLite format by using a set of training windows as a dataset that is representative in finding the proper scales and zero points for activations of each model.

So, pruning prunes 80% of parameters in dense layers whereas int8 quantization reduces 4 times model memory footprint, enabling the model's execution with integer kernels, resulting in a model that fits onto an Arduino Nano 33 BLE Sense.

3.

```
Final prediction: Lying down
Class index: 2
Confidence score: 0.6680
-----
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.8672, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.1328, 0.0000
Standard-scaled model scores: 0.0000, 0.1133, 0.4727, 0.0000, 0.2930, 0.0000, 0.0000, 0.0039, 0.0000, 0.0000, 0.0000, 0.1133
Min-max-scaled model scores: 0.0000, 0.0000, 0.1719, 0.2109, 0.1289, 0.0234, 0.4648, 0.0039, 0.0000, 0.0000, 0.0000, 0.0000
Meta-classifier scores: 0.0000, 0.1289, 0.6680, 0.0469, 0.0039, 0.0391, 0.0000, 0.0039, 0.0117, 0.0039, 0.0508, 0.0430
-----
```

A short explanation of whether the output is acceptable for the resting board condition: With the board lying flat, the algorithm outputs the class “Lying down”, with an accuracy rate of around 0.67. The prediction is correct, however considering the board is not moving at all, the confidence score should be higher.

Any issues you observe, such as unstable predictions, unexpected labels, sensor noise, scaling mismatch, or mismatch between the training data and the real board orientation: Based on the branch scores obtained from printing, each branch allocates its most probable distribution in terms of a few classes, however, this distribution varies depending on whether the branch is in its raw, standardized, or min-max format. For instance, in terms of the raw branch, there is much confidence allocated to one class (approximately 0.86 at index 2). On the other hand, the other branches allocate the same probability mass among several nearby classes. This might be a sign of a slight difference between the orientation/sensor units in the training data set and the actual Nano 33 BLE Sense IMU.

One or two practical changes that could improve deployment reliability:

Time prediction smoothing: Rather than having a single window-based approach, the use of an average or majority vote from the meta-classifier predictions for a small number of successive windows will help prevent any kind of flickering effect on the screen.

Fine-tuning the classifier according to the characteristics of the actual board: Collect a few extra IMU samples using the Nano 33 BLE Sense board as it is supposed to be placed and adjust the classifier itself or its scale/rotation to make the readings of your actual board match those of the mHealth sample.

4.

There was an inconsistency identified in terms of the model predictions after being implemented using the Arduino Nano 33 BLE Sense device. In spite of different activities being carried out by the participant, sometimes there was no difference in the prediction of the same state classes, which did not represent the performed action.

This phenomenon might be explained by multiple reasons. One of the possible reasons might be the discrepancy in features that were used in relation to those of the training dataset. The first mHealth dataset contains features extracted by the phone sensor located differently. As a result, movements recorded for training were less organized and sophisticated than those performed.